# Low-Latency Java & Systems Engineering

Interview roadmap tailored to the Senior Software Engineer - Low Latency role

**Your starting point:** experienced Java/Spring Boot engineer with production microservices experience, but limited direct low-latency/systems-performance specialization.

**Goal:** build enough conceptual depth and hands-on familiarity to interview credibly, then continue toward serious performance engineering.

## How to use this roadmap

- Follow the phases in order. Do not start with NUMA or Solarflare.
- For every major topic, learn the mechanism and then run a small experiment, benchmark, or diagnostic exercise.
- The core of this role is Java concurrency + JVM + networking + Linux + CPU/memory behavior + measurement.
- Do not try to become an HFT expert before the interview. Aim first for strong mental models and the ability to reason about latency.

**Suggested intensity:** 2-3 focused hours/day. The first 10 days are interview-oriented; the full roadmap can be continued over 8-10 weeks.

# 1. Mental Model: What Are You Optimizing?

Low latency is the discipline of reducing response time and, especially, reducing unpredictable delay (jitter and tail latency). The important part is understanding the entire path rather than one framework.

**Network NIC -> Linux networking -> TCP/UDP -> Java NIO/socket -> Java threads -> concurrent structures -> JVM/JIT/GC -> virtual memory -> CPU caches -> cores/NUMA/TLB -> hardware**

## Key metrics

- Latency: time from input/event/request to the desired output.
- Throughput: amount of work completed per unit time.
- Tail latency: p95, p99, p99.9, etc. A good average can hide bad tails.
- Jitter: variation in latency between operations.
- Contention: multiple threads/resources competing for a shared resource.
- Saturation: the point where increasing load causes latency to rise sharply.

Interview principle: if asked how to make a service faster, first measure and locate the bottleneck. Do not immediately guess at an optimization.

# 2. Priority Map

| Priority | Topic | Reason |
|---|---|---|
| P0 | Java concurrency + memory model | Explicit requirement; central to shared-memory performance. |
| P0 | JVM, JIT, allocation, GC | Explicit requirement; important for latency spikes. |
| P0 | Benchmarking / profiling | You must prove an optimization actually helped. |
| P0 | TCP/UDP + Java NIO | Explicit requirement; understand the network path. |
| P0 | Linux processes/threads | Explicit advanced requirement. |
| P1 | CPU cache + false sharing | Explains many low-level latency effects. |
| P1 | Queues + backpressure + Little's Law | Explicit queue-related requirement. |
| P1 | perf / tcpdump / JFR | Core diagnostic tools. |
| P2 | NUMA / affinity / core isolation / TLB | Preferred qualifications; specialized. |
| P2 | JNI / Solarflare / EF_VI | Vendor/native technologies; learn last. |

# 3. Phase 1 - Java Concurrency & Java Memory Model

**Target:** explain races, visibility, atomicity, locks, CAS, concurrent collections, queues, contention and the Java Memory Model.

**Primary book:** Java Concurrency in Practice - Brian Goetz et al. Official contents

## Read first

- Ch. 1 - Introduction: skim.
- Ch. 2 - Thread Safety: atomicity, locking, liveness.
- Ch. 3 - Sharing Objects: visibility, publication, immutability, safe publication.
- Ch. 4 - Composing Objects: thread-safe class design.
- Ch. 5 - Building Blocks: ConcurrentHashMap, blocking queues, synchronizers.
- Ch. 6 - Task Execution: Executor framework and parallelism.
- Ch. 8 - Thread Pools: sizing and execution policies.
- Ch. 10 - Liveness Hazards: deadlock and diagnosis.
- Ch. 11 - Performance and Scalability: Amdahl's law, contention, context switches.
- Ch. 12 - Testing Concurrent Programs.
- Ch. 13 - Explicit Locks.
- Ch. 15 - Atomic Variables and Nonblocking Synchronization: CAS.
- Ch. 16 - Java Memory Model.

## Hands-on

- Break a shared counter with multiple threads; reproduce lost updates.
- Fix it using synchronized, AtomicLong, and LongAdder; explain the tradeoffs.
- Build a producer-consumer pipeline with BlockingQueue and measure queue depth/latency.
- Create and diagnose a deadlock using thread dumps.
- Create a false-sharing benchmark with two counters, then separate/pad them.

Why this matters: this phase covers the largest overlap between the job description and your existing Java skill set. JCIP contents

# 4. Phase 2 - Benchmarking, Profiling & JVM Performance

**Target:** measure latency correctly, understand JVM warm-up/JIT/allocation/GC, and explain why naive benchmarks can mislead you.

## JMH - Java Microbenchmark Harness

Use JMH for Java microbenchmarks instead of hand-written timing loops. JMH is designed for JVM benchmarks and includes samples. OpenJDK JMH

- Run the JMH samples first.
- Benchmark HashMap vs ConcurrentHashMap for an appropriate workload.
- Benchmark synchronized vs AtomicLong vs LongAdder.
- Compare allocation-heavy and allocation-light versions of the same operation.
- Learn warmup, forks, iterations, throughput vs average time, and why benchmark isolation matters.

## JVM topics

- Heap, thread stacks, metaspace/class metadata.
- Object allocation and object lifetime.
- JIT compilation and warm-up.
- Escape analysis at a conceptual level.
- Safepoints and runtime events.
- Garbage collection: generations, allocation rate, pauses, concurrent work.
- Latency vs throughput tradeoffs when selecting/tuning a collector.

## GC resources

Start with Oracle GC tuning; then read the latency sections in G1 GC tuning.

## Java Flight Recorder

Learn events, recordings and diagnostic commands. Oracle JFR runtime guide

**Mini-project:** create a Java load generator, record p50/p95/p99, compare an allocation-heavy implementation with an allocation-light implementation, and use JFR to explain the difference.

# 5. Phase 3 - Java NIO, Networking, TCP & UDP

**Target:** understand the path from NIC to Java and why blocking I/O, buffering, copies, kernel transitions, retransmissions and protocol behavior affect latency.

## Java NIO

Focus on ByteBuffer, Channel, SocketChannel, ServerSocketChannel, Selector and SelectionKey. Oracle describes buffers, channels and selectors as central NIO abstractions, with selectors enabling multiplexed non-blocking I/O. Java NIO API

**Build:** a Java echo server. First use blocking sockets; then implement a selector-based non-blocking version. Explain why the architecture changes.

## TCP vs UDP

- TCP connection establishment, reliability and ordering.
- Retransmission, congestion control and flow control.
- UDP's lack of built-in reliability/ordering and why an application may prefer it.
- Sockets, ports, packets, MTU, loss, retransmission and jitter.
- Basic Linux kernel/network-stack path.

## Tools

- tcpdump - packet capture from the command line.
- Wireshark - inspect packet timing/protocol behavior.
- ss - inspect sockets and connection state.
- Basic ping/path diagnostics.

**Interview exercise:** a service is usually 200 microseconds but occasionally 5 milliseconds. Walk through a systematic investigation: application timing -> JFR/profiling -> packet capture -> retransmission -> GC/scheduler/CPU checks -> isolate the tail-latency source.

# 6. Phase 4 - Linux, Operating Systems & Performance Tools

**Target:** processes, threads, scheduling, virtual memory, system calls, interrupts, CPU affinity and Linux performance tooling.

**Primary free book:** Operating Systems: Three Easy Pieces (OSTEP). It is organized around virtualization, concurrency and persistence and is available chapter-by-chapter. OSTEP

## Read selectively

■ Processes (Ch. 5) and Direct Execution (Ch. 6).

■ CPU Scheduling (Ch. 7) and Multi-CPU Scheduling (Ch. 10).

■ Address Spaces (Ch. 13) and Address Translation (Ch. 15).

■ Translation Lookaside Buffers (Ch. 19).

■ Concurrency and Threads (Ch. 26), Thread API (27), Locks (28), Locked Data Structures (29), Condition Variables (30), Semaphores (31), Concurrency Bugs (32), Event-based Concurrency (33).

■ I/O Devices (Ch. 36).

## Linux performance tools

Brendan Gregg's Linux Performance material explains performance methodology plus tools such as top, iostat, tcpdump, perf, ftrace and tracing/profiling concepts. Linux Performance

**Practice:** top/htop, ps, vmstat, iostat, ss, tcpdump, perf stat, perf record/report, taskset, jcmd and jstack.

**Exercise:** run a CPU-intensive Java process, inspect CPU utilization and thread placement, use perf where available, then pin a process with taskset and observe what changes. The objective is measurement, not guaranteed improvement.

# 7. Phase 5 - CPU Caches, Memory, Queues & Latency Theory

**Target:** understand why two algorithms with similar Big-O complexity can have very different real latency.

## CPU / memory topics

- CPU cores and hardware threads.
- L1/L2/L3 caches and cache locality.
- Cache lines and cache coherence.
- False sharing.
- Branch prediction and branch misprediction.
- RAM vs cache latency.
- Virtual memory and TLBs.
- Basic NUMA concepts.

## Queues and queue theory

- FIFO, bounded/unbounded queues and producer-consumer design.
- Backpressure and why unbounded queues can hide a growing latency problem.
- Blocking vs non-blocking/concurrent queues.
- Contention and queueing delay.
- Little's Law: L = lambda * W.
- Why latency can rise sharply as utilization approaches saturation.

## LMAX Disruptor - especially relevant here

Read the LMAX Disruptor documentation. It was developed from work on a high-performance financial exchange and discusses queueing, contention, latency, jitter, CPU timing and performance testing. Disruptor documentation

**Project:** implement a bounded ring buffer in Java and benchmark it against ArrayBlockingQueue and ConcurrentLinkedQueue under controlled producer/consumer workloads. Report p50/p99 and explain contention/cache effects.

# 8. Phase 6 - Specialized Low-Latency Topics

Study these only after the foundations. They are preferred/specialized qualifications in the posting.

| Topic | What you should know |
|---|---|
| Thread pinning / CPU affinity | How to keep latency-sensitive threads on selected cores; basic affinity tooling. |
| Core isolation | Reserve CPUs to reduce interference from unrelated work. |
| Interrupt handling | How hardware interrupts and interrupt affinity can introduce variability. |
| NUMA | Memory attached to different CPU nodes can have different access costs; placement matters. |
| TLB | Cache of virtual-to-physical address translations; misses can add memory-access cost. |
| Jitter reduction | Reduce GC pauses, scheduler interference, interrupts, contention and other sources of variability. |
| JNI | Java/native boundary; why native code may be used and its costs/complexity. |
| Solarflare Onload | Specialized high-performance networking; understand the motivation before API details. |
| EF_VI | Specialized high-performance NIC interface; conceptual knowledge is enough initially. |

# 9. 10-Day Interview Sprint

If the client interview is soon, use this compressed sequence. Do not try to finish the whole roadmap.

| Day | Focus | Deliverable |
|---|---|---|
| 1 | Concurrency | Threads, races, atomicity, locks; broken/fixed counter. |
| 2 | JMM | Visibility, volatile, happens-before, CAS, safe publication. |
| 3 | Concurrent structures | ConcurrentHashMap, queues, producer-consumer, contention. |
| 4 | JVM/GC | Heap, allocation, JIT, safepoints, GC and tail latency. |
| 5 | JMH/JFR | 3-4 benchmarks; warmup; p99; basic JFR investigation. |
| 6 | NIO/networking | Selector-based echo server; TCP vs UDP. |
| 7 | Linux/OS | Processes, threads, scheduling, virtual memory; perf/top/vmstat. |
| 8 | CPU/memory | Caches, cache lines, false sharing, TLB, NUMA concepts. |
| 9 | Queues/Disruptor | Little's Law, backpressure, ring buffer, latency/jitter. |
| 10 | Mock interview | End-to-end latency diagnosis + concurrency/system questions. |

# 10. 8-10 Week Deeper Roadmap

| Weeks | Goal |
|---|---|
| 1-2 | Java concurrency + JMM; selected JCIP chapters and experiments. |
| 3 | JMH, JFR, JVM internals, allocation, GC, profiling. |
| 4 | Java NIO, sockets, TCP/UDP, packet capture, non-blocking I/O. |
| 5 | OS fundamentals with OSTEP: processes, scheduling, virtual memory, threads. |
| 6 | Linux performance: perf, tcpdump, vmstat, iostat, CPU/thread diagnostics. |
| 7 | CPU architecture: caches, coherence, false sharing, branch prediction, TLB. |
| 8 | Queues, Little's Law, backpressure, ring buffers, Disruptor. |
| 9-10 | NUMA, affinity, core isolation, interrupts, JNI, Solarflare/EF_VI concepts; final benchmark. |

# 11. Curated Resource Index

## Books

- **Java Concurrency in Practice** - strongest resource for the Java concurrency portion. Prioritize Ch. 2-6, 8, 10-13, 15-16. Contents
- **Operating Systems: Three Easy Pieces** - free online; use it selectively for processes, scheduling, memory, TLBs and concurrency. Book
- **Computer Organization & Design** - use selectively for CPU/memory architecture if caches/TLB/NUMA remain abstract.

## Code / repositories

- **OpenJDK JMH** - benchmark framework plus samples. GitHub
- **LMAX Disruptor** - high-performance concurrent structure and documentation grounded in financial exchange systems. GitHub

## Documentation / practical references

- **Java NIO API** - buffers, channels, selectors and non-blocking I/O. Oracle
- **Oracle GC Tuning** - collector behavior and tuning. Oracle
- **G1 GC Tuning** - latency-related tuning sections. Oracle
- **Brendan Gregg - Linux Performance** - methodology, observability, benchmarking, tuning and tracing. Website

# 12. Do NOT Overstudy Yet

- Solarflare Onload API details.
- EF_VI API details.
- Deep Linux kernel source internals.
- Writing a custom JVM.
- Advanced lock-free algorithms beyond what you need for CAS/Disruptor.
- Detailed CPU microarchitecture manuals.
- Every GC and every JVM flag.

**Rule:** understand the mechanism first. Memorize vendor-specific APIs only if the interview specifically targets them.

# 13. Final Interview Checklist

- ■ Explain latency vs throughput vs tail latency vs jitter.
- ■ Explain and fix a race condition.
- ■ Explain volatile vs synchronized vs AtomicLong.
- ■ Explain CAS and why lock-free structures can help.
- ■ Explain ConcurrentHashMap conceptually.
- ■ Explain producer-consumer, queue contention and backpressure.
- ■ Explain happens-before / the Java Memory Model.
- ■ Explain why GC can create latency spikes.
- ■ Explain why JMH is preferable to naive timing.

■ Explain Java NIO selectors and non-blocking I/O.

■ Explain TCP vs UDP and retransmission.

■ Use basic Linux performance tools and explain what each tells you.

■ Explain cache lines and false sharing.

■ Explain NUMA, TLB, thread pinning and core isolation conceptually.

■ Describe a systematic approach to diagnosing a p99 latency regression.

**Recommended capstone:** build a small market-data-style Java pipeline: UDP/TCP input -> NIO/network layer -> parser -> concurrent queue/ring buffer -> worker -> result. Benchmark with JMH, record p50/p95/p99, profile with JFR, inspect the process with Linux tools, and write a short performance report. This ties together nearly every major topic in the job description.